

WIRESHARK is a tool used to monitor and analyze network traffic.

Documentation

Virtual Machines on Virtualbox

- Windows 11 VM (Wireshark workstation)
- Ubuntu Linux server
- windows server 2022
- Kali Linux

Tools:

- Wireshark
- Command Prompt
- Ping
- SSH

STEP 1

Kali IP: 192.168.56.103

Ubuntu IP: 192.168.56.102

Ping test = success

Install wireshark on kali

```
Sudo apt update
```

```
Sudo apt install wireshark -y
```

After installing wireshark neither eth0 or eth1 show traffic lets check id wireshark has permission to capture packets

- Lets check what interface kali has
- Looks like kali has 2 interfaces both eth0 and eth1 (I used NAT and host-only/internal network)

Force traffic on eth1 by double clicking

Tested pinging Ubuntu on Kali command and was able to see ICMP echo request and echo reply, ARP broadcast, and DHCP ACK captures

Analyze an Echo request packet

Source: 192.168.56.103 Kali

Destination: 192.168.56.102 Ubuntu
Type: Echo (ping) request (8)
Code: 0

Different types and codes mean
Type 8 = request
Type 0 = reply
Code 0 = normal ping

Wireshark packet capture showing network traffic. The interface is 'Capturing from eth1'. The packet list shows:

No.	Time	Source	Destination	Protocol	Length	Info
18	30.389640376	192.168.56.103	192.168.56.102	ICMP	98	Echo (ping) request id=0x37
19	30.389910306	192.168.56.102	192.168.56.103	ICMP	98	Echo (ping) reply id=0x37
20	31.413380475	192.168.56.103	192.168.56.102	ICMP	98	Echo (ping) request id=0x37
21	31.413658138	192.168.56.102	192.168.56.103	ICMP	98	Echo (ping) reply id=0x37
22	32.437517734	192.168.56.103	192.168.56.102	ICMP	98	Echo (ping) request id=0x37
23	32.438323019	192.168.56.102	192.168.56.103	ICMP	98	Echo (ping) reply id=0x37
24	198.974502099	192.168.56.102	192.168.56.100	DHCP	340	DHCP Request - Transaction ID 0x315bf
25	198.983342277	PCSSystemtec_3f:49:...	Broadcast	ARP	60	Who has 192.168.56.102? Tell 192.168.56.100
26	198.983342692	192.168.56.100	192.168.56.102	DHCP	590	DHCP ACK - Transaction ID 0x315bf
27	198.983438392	PCSSystemtec_b5:cb:...	PCSSystemtec_3f:49:...	ARP	60	192.168.56.102 is at 08:00:27:b5:cb:bd
28	203.975985374	PCSSystemtec_b5:cb:...	PCSSystemtec_3f:49:...	ARP	60	Who has 192.168.56.100? Tell 192.168.56.100
29	203.975987037	PCSSystemtec_3f:49:...	PCSSystemtec_b5:cb:...	ARP	60	192.168.56.100 is at 08:00:27:3f:49:8d
30	214.493920303	fe80::4133:5252:c0d...	ff02::c	UDP/XML	718	56782 - 3702 Len=656
31	214.494439916	192.168.56.1	239.255.255.250	UDP/XML	698	56781 - 3702 Len=656
32	214.649546288	192.168.56.1	239.255.255.250	UDP/XML	698	56781 - 3702 Len=656
33	214.742291863	fe80::4133:5252:c0d...	ff02::c	UDP/XML	718	56782 - 3702 Len=656
34	214.957818184	192.168.56.1	239.255.255.250	UDP/XML	698	56781 - 3702 Len=656

Frame 26: Packet, 590 bytes on wire (4720 bits), 590 bytes captured (4720 bits) on interface eth1, 0 bytes from 192.168.56.100. Ethernet II, Src: PCSSystemtec_3f:49:8d (08:00:27:3f:49:8d), Dst: 192.168.56.100 (01:00:00:00:00:00), Internet Protocol Version 4, Src: 192.168.56.100, Destination: 192.168.56.102, User Datagram Protocol, Src Port: 67, Dst Port: 68, Dynamic Host Configuration Protocol (ACK).

LLMNR = windows machine use it for local name resolution
The LLMNR captures show that kali and the window machine are already communicating. I want to make sure of the Ip address of the windows 11 (192.168.56.101)

	Source	Destination	Protocol	Length	Info
7029293	192.168.56.101	224.0.0.22	IGMPv3	60	Membership Report / Leave group 224.0.0.22
7288664	fe80::e34a:ef2d:f7a...	ff02::16	ICMPv6	90	Multicast Listener Report Message v2
7288812	192.168.56.101	224.0.0.22	IGMPv3	60	Membership Report / Join group 224.0.0.22
8206319	fe80::e34a:ef2d:f7a...	ff02::1:3	LLMNR	88	Standard query 0xf1d7 ANY DESKTOP1
8283671	192.168.56.101	224.0.0.252	LLMNR	68	Standard query 0xf1d7 ANY DESKTOP1
4284634	192.168.56.101	224.0.0.22	IGMPv3	60	Membership Report / Join group 224.0.0.22
4285062	fe80::e34a:ef2d:f7a...	ff02::16	ICMPv6	90	Multicast Listener Report Message v2
9183317	fe80::e34a:ef2d:f7a...	ff02::1:3	LLMNR	88	Standard query 0xf1d7 ANY DESKTOP1
9183964	192.168.56.101	224.0.0.252	LLMNR	68	Standard query 0xf1d7 ANY DESKTOP1
5493877	192.168.56.102	192.168.56.100	DHCP	340	DHCP Request - Transaction ID 0x315bf
7209256	PCSSystemtec_3f:49:...	Broadcast	ARP	60	Who has 192.168.56.102? Tell 192.168.56.100
7209569	192.168.56.100	192.168.56.102	DHCP	590	DHCP ACK - Transaction ID 0x315bf
7209620	PCSSystemtec_b5:cb:...	PCSSystemtec_3f:49:...	ARP	60	192.168.56.102 is at 08:00:27:b5:cb:bd
5489304	PCSSystemtec_41:03:...	PCSSystemtec_3f:49:...	ARP	60	Who has 192.168.56.100? Tell 192.168.56.100
5489622	PCSSystemtec_3f:49:...	PCSSystemtec_41:03:...	ARP	60	192.168.56.100 is at 08:00:27:3f:49:8d
9324989	PCSSystemtec_b5:cb:...	PCSSystemtec_3f:49:...	ARP	60	Who has 192.168.56.100? Tell 192.168.56.100
9325462	PCSSystemtec_3f:49:...	PCSSystemtec_b5:cb:...	ARP	60	192.168.56.100 is at 08:00:27:3f:49:8d

Applying a filter
ARP

_ws.col.protocol == "ARP"						
No.	Time	Source	Destination	Protocol	Length	Info
2	0.006889552	PCSSystemtec_3f:49:...	Broadcast	ARP	60	Who has 192.168.56.101? Tell
4	0.007206924	PCSSystemtec_41:03:...	PCSSystemtec_3f:49:...	ARP	60	192.168.56.101 is at 08:00:27
20	3.397209256	PCSSystemtec_3f:49:...	Broadcast	ARP	60	Who has 192.168.56.102? Tell
22	3.397209620	PCSSystemtec_b5:cb:...	PCSSystemtec_3f:49:...	ARP	60	192.168.56.102 is at 08:00:27
23	4.575489304	PCSSystemtec_41:03:...	PCSSystemtec_3f:49:...	ARP	60	Who has 192.168.56.100? Tell
24	4.575489622	PCSSystemtec_3f:49:...	PCSSystemtec_41:03:...	ARP	60	192.168.56.100 is at 08:00:27
25	8.619324989	PCSSystemtec_b5:cb:...	PCSSystemtec_3f:49:...	ARP	60	Who has 192.168.56.100? Tell
26	8.619325462	PCSSystemtec_3f:49:...	PCSSystemtec_b5:cb:...	ARP	60	192.168.56.100 is at 08:00:27

I did a **nmap** scan for open ports on the Ubuntu server.

Results: 999 closed tcp ports

22/ Tcp open ssh

```
(kali@kali)-[~]
$ nmap -A 192.168.56.102
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-01 11:52 -0400
Nmap scan report for 192.168.56.102
Host is up (0.00044s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 (Ubuntu Linux; protocol 2.0)
MAC Address: 08:00:27:B5:CB:BD (Oracle VirtualBox virtual NIC)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ )
TCP/IP fingerprint:
OS:SCAN(V=7.98%E=4%D=6/1%OT=22%CT=1%CU=41169%PV=Y%DS=1%DC=D%G=Y%M=080027%TM
OS:=6A1DAAE7P=x86_64-pc-linux-gnu)SEQ(SP=102%GCD=1%ISR=107%TI=Z%CI=Z%TS=21
OS: )SEQ(SP=102%GCD=2%ISR=10D%TI=Z%CI=Z%II=I%TS=21)SEQ(SP=104%GCD=1%ISR=106%
OS:TI=Z%CI=Z%II=I%TS=22)SEQ(SP=104%GCD=1%ISR=10B%TI=Z%CI=Z%II=I%TS=21)SEQ(S
OS:P=FD%GCD=1%ISR=106%TI=Z%CI=Z%II=I%TS=22)OPS(O1=M5B4ST11NWA%O2=M5B4ST11NW
OS:A%O3=M5B4NNT11NWA%O4=M5B4ST11NWA%O5=M5B4ST11NWA%O6=M5B4ST11)WIN(W1=FE88%
OS:W2=FE88%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M5B4N
OS:NSNWA%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=0%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=
OS:Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=A
OS:R%O=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=4
OS:0%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=
OS:G%RIPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
```

Wireshark can see the network conversation, it cannot read your password or commands because SSH encrypts everything.

It will show the 3-way handshake.

Time	Source	Destination	Protocol	Length	Info
33.28.208247266	PCSSystemtec_b5:cb:...	PCSSystemtec_5c:cf:...	ARP	60	Who has 192.168.56.103? Tell 192.168.56.102
34.28.208261897	PCSSystemtec_5c:cf:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.103 is at 08:00:27:5c:cf:6a
35.28.679265215	192.168.56.103	192.168.56.102	SSHv2	214	Client: Encrypted packet (len=148)
36.28.698329764	192.168.56.102	192.168.56.103	SSHv2	94	Server: Encrypted packet (len=28)
37.28.698359222	192.168.56.103	192.168.56.102	TCP	66	45398 → 22 [ACK] Seq=3186 Ack=2999 Win=64512 Len=0 TSval=2629...
38.28.698686221	192.168.56.103	192.168.56.102	SSHv2	178	Client: Encrypted packet (len=112)
39.28.739666001	192.168.56.102	192.168.56.103	TCP	66	22 → 45398 [ACK] Seq=2999 Ack=3298 Win=64512 Len=0 TSval=2293...
40.29.372725302	192.168.56.102	192.168.56.103	SSHv2	694	Server: Encrypted packet (len=628)
41.29.372843154	192.168.56.102	192.168.56.103	SSHv2	110	Server: Encrypted packet (len=44)
42.29.372931119	192.168.56.103	192.168.56.102	TCP	66	45398 → 22 [ACK] Seq=3298 Ack=3671 Win=64512 Len=0 TSval=2629...
43.29.373293446	192.168.56.103	192.168.56.102	SSHv2	594	Client: Encrypted packet (len=528)
44.29.373522763	192.168.56.102	192.168.56.103	TCP	66	22 → 45398 [ACK] Seq=3671 Ack=3826 Win=64512 Len=0 TSval=2293...
45.29.374535316	192.168.56.102	192.168.56.103	SSHv2	174	Server: Encrypted packet (len=108)
46.29.376150586	192.168.56.102	192.168.56.103	SSHv2	1246	Server: Encrypted packet (len=1180)
47.29.376263700	192.168.56.103	192.168.56.102	TCP	66	45398 → 22 [ACK] Seq=3826 Ack=4959 Win=64512 Len=0 TSval=2629...
48.29.514576324	192.168.56.102	192.168.56.103	SSHv2	446	Server: Encrypted packet (len=380)
49.29.555811212	192.168.56.103	192.168.56.102	TCP	66	45398 → 22 [ACK] Seq=3826 Ack=5339 Win=64512 Len=0 TSval=2629...

Frame 42: Packet, 66 bytes on wire (528 bits), 66 bytes captured (5	0000	08 00 27 b5 cb bd 08 00	27 5c cf 6a 08 00 45 b8	...
Ethernet II, Src: PCSSystemtec_5c:cf:6a (08:00:27:5c:cf:6a), Dst: P	0010	00 34 6b c2 40 00 40 06	dc 2b c0 a8 38 67 c0 a8	4k @ @ +
Internet Protocol Version 4, Src: 192.168.56.103, Dst: 192.168.56.1	0020	38 66 b1 56 00 16 a0 e5	11 0d 32 1b 88 3f 80 10	8f V ...
Transmission Control Protocol, Src Port: 45398, Dst Port: 22, Seq:	0030	00 fc f2 44 00 00 01 01	08 0a 9c bf ea 01 88 b8	... D ...
	0040	4c df		L

DNS

Looking for domains in wireshark

3-WAY handshake

[SYN] client sends to server

[SYN, ACK] server sends back that it has received and acknowledged.

[ACK] client acknowledges as well.

[PSH] DATA

[FIN] connection is closed

To capture DNS traffic

I did nslookup openai.com or pinging openai.com

0	120.163137864	10.0.2.15	192.168.1.254	DNS	70 Standard query 0x6677 A openai.com
1	120.184561714	192.168.1.254	10.0.2.15	DNS	192 Standard query response 0x6677 A openai.com A 172.64.154.211
2	120.185457165	10.0.2.15	192.168.1.254	DNS	70 Standard query 0x6fe8 AAAA openai.com
3	120.202655315	192.168.1.254	10.0.2.15	DNS	153 Standard query response 0x6fe8 AAAA openai.com SOA ns1-02.azu.

I look like all these captures belong to RIPE network coordination Centre (large hosting/CDN network)

Notice the [ACK]

- Vm INITIATED THE CONNECTION
- Port 443 = HTTPS
- Data is encrypted
- This is normal web traffic

273	83.788448639	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1519 Ack=72848 Win=65535 Len=0
275	83.788613898	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1519 Ack=74204 Win=65535 Len=0
277	83.788669915	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1519 Ack=75560 Win=65535 Len=0
279	83.788969610	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1519 Ack=76916 Win=65535 Len=0
281	83.789053785	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1519 Ack=78272 Win=65535 Len=0
283	83.789167375	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1519 Ack=79628 Win=65535 Len=0
285	83.789255244	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1519 Ack=80984 Win=65535 Len=0
287	83.789405756	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1519 Ack=82340 Win=65535 Len=0
290	83.789490437	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1519 Ack=83780 Win=65535 Len=0
291	83.789510935	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1519 Ack=83782 Win=65535 Len=0
292	85.813428182	10.0.2.15	146.75.93.91	TLSv1.2	196 Application Data
295	85.819630447	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1661 Ack=84135 Win=65535 Len=0
299	85.820941790	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1661 Ack=85491 Win=65535 Len=0
300	85.821039007	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1661 Ack=86931 Win=65535 Len=0
301	85.821234818	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1661 Ack=88203 Win=65535 Len=0
304	85.821287353	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1661 Ack=89643 Win=65535 Len=0
305	85.821304091	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1661 Ack=90129 Win=65535 Len=0
306	86.796712702	10.0.2.15	146.75.93.91	TLSv1.2	178 Application Data
310	86.805322160	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=91569 Win=65535 Len=0
311	86.805366082	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=92841 Win=65535 Len=0
313	86.805484853	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=94197 Win=65535 Len=0
316	86.805697228	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=95637 Win=65535 Len=0
317	86.805731809	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=96909 Win=65535 Len=0
319	86.805790566	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=98265 Win=65535 Len=0
323	86.806054503	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=99705 Win=65535 Len=0
324	86.806065015	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=101145 Win=65535 Len=0
325	86.806070570	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=102333 Win=65535 Len=0
327	86.806221482	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=103689 Win=65535 Len=0
330	86.806460614	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=105129 Win=65535 Len=0
331	86.806471391	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=106006 Win=65535 Len=0
334	86.806635449	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=107446 Win=65535 Len=0
335	86.806646122	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=108718 Win=65535 Len=0
344	86.810450585	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=110158 Win=65535 Len=0
345	86.810463905	10.0.2.15	146.75.93.91	TCP	54 58208 → 443 [ACK] Seq=1785 Ack=111598 Win=65535 Len=0

How would an attack look like

- Many [SYN] packets
- Many destination ports
- Many destination IPs.
- Attacker could try to reach (22, 80, 443, 3349, 445)

SSH session showing the 3-way handshake between client and server.

517	201.242227369	192.168.56.103	192.168.56.102	TCP	74 59884 → 22 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=251530...
518	201.242624270	192.168.56.102	192.168.56.103	TCP	74 22 → 59884 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM T...
519	201.242651389	192.168.56.103	192.168.56.102	TCP	66 59884 → 22 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=251530838 TSecr=371...
520	201.242889279	192.168.56.103	192.168.56.102	SSHv2	99 Client: Protocol (SSH-2.0-OpenSSH_10.2p1 Debian-5)

This is also helpful to see if the connection succeeded

If missing the 3-way handshake

- Firewall issue
- Host down
- Packet drop

Finding open ports

On KALI

```
Nmap -sS 192.168.56.102
```

This doesn't show the 3-way handshake, this scans for open ports, by checking each port.

56075 → 4443 [SYN]

56075 → 9000 [SYN]

56075 → 3784 [SYN]

56075 → 4848 [SYN]

This shows that everything is closed

[RST] = REST, NO SERVICE HERE

2708	399.878469382	192.168.56.103	192.168.56.102	TCP	58 56075 → 5432 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2709	399.878483588	192.168.56.102	192.168.56.103	TCP	60 1020 → 56075 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2710	399.878483678	192.168.56.102	192.168.56.103	TCP	60 4567 → 56075 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2711	399.878483720	192.168.56.102	192.168.56.103	TCP	60 2170 → 56075 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2712	399.878506341	192.168.56.103	192.168.56.102	TCP	58 56075 → 32782 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2713	399.878513909	192.168.56.103	192.168.56.102	TCP	58 56075 → 1034 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2714	399.878520524	192.168.56.103	192.168.56.102	TCP	58 56075 → 7625 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2715	399.878527021	192.168.56.103	192.168.56.102	TCP	58 56075 → 5922 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2716	399.878533248	192.168.56.103	192.168.56.102	TCP	58 56075 → 7999 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2717	399.878539278	192.168.56.103	192.168.56.102	TCP	58 56075 → 5963 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2718	399.878545328	192.168.56.103	192.168.56.102	TCP	58 56075 → 5225 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2719	399.878551333	192.168.56.103	192.168.56.102	TCP	58 56075 → 2041 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2720	399.878559318	192.168.56.103	192.168.56.102	TCP	58 56075 → 55600 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2721	399.878603781	192.168.56.103	192.168.56.102	TCP	58 56075 → 7002 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
2722	399.878649482	192.168.56.102	192.168.56.103	TCP	60 5432 → 56075 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2723	399.878693210	192.168.56.102	192.168.56.103	TCP	60 32782 → 56075 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2724	399.878693292	192.168.56.102	192.168.56.103	TCP	60 1034 → 56075 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
2725	399.878693341	192.168.56.102	192.168.56.103	TCP	60 7625 → 56075 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0

Finding open ports

```
Sudo nmap -p 22 192.168.56.102
```

You can see the 3-way handshake.= open port

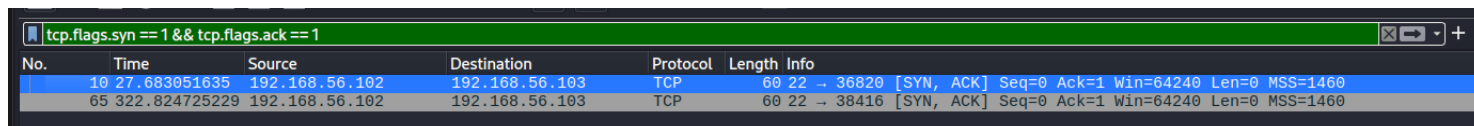
8	27.074418382	PCSSystemtec_b5:cb:...	PCSSystemtec_5c:cf:...	ARP	60 192.168.56.102 is at 08:00:27:b5:cb:bd
9	27.682417340	192.168.56.103	192.168.56.102	TCP	58 36820 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460
10	27.683051635	192.168.56.102	192.168.56.103	TCP	60 22 → 36820 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
11	27.683088804	192.168.56.103	192.168.56.102	TCP	54 36820 → 22 [RST] Seq=1 Win=0 Len=0
12	27.705791524	PCSSystemtec_b5:cb:...	PCSSystemtec_5c:cf:...	ARP	60 192.168.56.102 is at 08:00:27:b5:cb:bd

Finding open ports through a full scan by filter

```
Nmap -sS 192.168.56.102
```

Apply this filter

```
tcp.flags.syn == 1 && tcp.flags.ack == 1
```



No.	Time	Source	Destination	Protocol	Length	Info
10	27.683051635	192.168.56.102	192.168.56.103	TCP	60	22 → 36820 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
65	322.824725229	192.168.56.102	192.168.56.103	TCP	60	22 → 38416 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460

Opening more ports to the public for the purpose of this lab only.

On Ubuntu machine

1. HTTP (PORT 80)

Sudo apt update

Sudo spt install apache2 -y

VERIFY

Sudo ss -tulnp | grep :80

```
connectionuser@connectioniot:~$ sudo ss -tulnp | grep :80
tcp    LISTEN 0      511          *:80         *:            users:(("apache2",pid=3039,fd=4),("apache2",pid=3038,fd=4),("apache2",pid=3022,fd=4))
```

On wireshark filter (http)

- GET requests
- HTTP responses
- Headers

2. HTTPS (port 443)

Sudo apt install openssl -y (install SSL support)

We can see on wireshark filter (tls)

- TLS handshake
- Certificates
- Encrypted web traffic

3. DNS (port 53)

Sudo apt install bind9 -y (install a dns server)

On wireshark filter (dns)

- Queries
- Responses
- Domain lookups

4. SMB (ports 445/139)

Sudo apt install samba -y (install samba)

On wireshark filter (smb2)

- File sharing
- Authentication
- Windows traffic

5. FTP (port 21)

Sudo apt install vsftpd -y

On wireshark filter (ftp)

Username and passwords in plaintext.

FTP is not a secure protocol.

HTTP

I searched up the IP address and everything on wireshark displays the type of {request method}, {the host}, {user-agent}: this is not a secure protocol

Results show

- TLS Handshake,
- Certificate exchange
- Source IP
- Destination IP
- Encrypted application data.

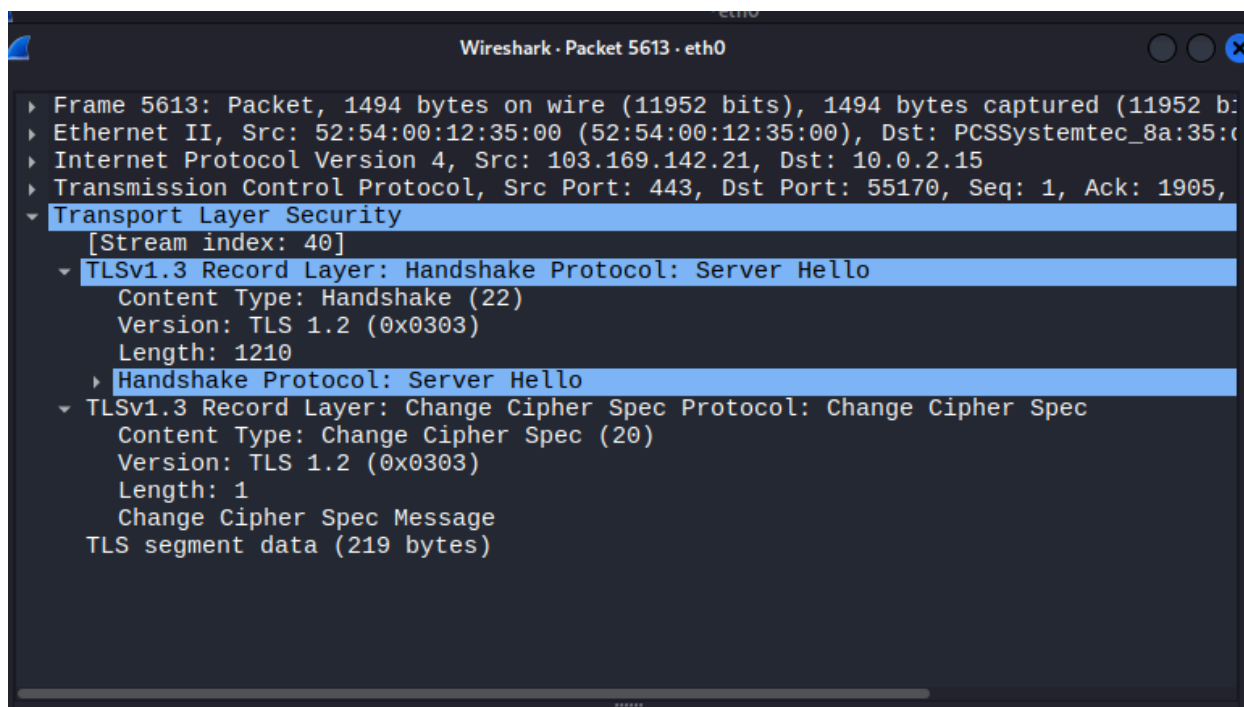
```
▼ Hypertext Transfer Protocol
  ▼ GET / HTTP/1.1\r\n
    Request Method: GET
    Request URI: /
    Request Version: HTTP/1.1
    Host: 192.168.56.102\r\n
    User-Agent: curl/8.18.0\r\n
    Accept: */*\r\n
    \r\n
```

1	0.000000000	192.168.56.103	192.168.56.102	TCP	74	52400 → 80	[SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2705337...
2	0.000342687	192.168.56.102	192.168.56.103	TCP	74	80 → 52400	[SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TS...
3	0.000365447	192.168.56.103	192.168.56.102	TCP	66	52400 → 80	[ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=270533795 TSecr=2335...
4	0.000463036	192.168.56.103	192.168.56.102	HTTP	144	GET / HTTP/1.1	
5	0.000709486	192.168.56.102	192.168.56.103	TCP	66	80 → 52400	[ACK] Seq=1 Ack=79 Win=65536 Len=0 TSval=2335246519 TSecr=27...
6	0.001174384	192.168.56.102	192.168.56.103	TCP	7306	80 → 52400	[PSH, ACK] Seq=1 Ack=79 Win=65536 Len=7240 TSval=2335246519 ...
7	0.001174502	192.168.56.102	192.168.56.103	HTTP	3753	HTTP/1.1 200 OK	(text/html)
8	0.001189137	192.168.56.103	192.168.56.102	TCP	66	52400 → 80	[ACK] Seq=79 Ack=7241 Win=79872 Len=0 TSval=270533796 TSecr=...
9	0.001198709	192.168.56.103	192.168.56.102	TCP	66	52400 → 80	[ACK] Seq=79 Ack=10928 Win=84992 Len=0 TSval=270533796 TSecr=...
10	0.001538948	192.168.56.103	192.168.56.102	TCP	66	52400 → 80	[FIN, ACK] Seq=79 Ack=10928 Win=84992 Len=0 TSval=270533797 ...
11	0.001943527	192.168.56.102	192.168.56.103	TCP	66	80 → 52400	[FIN, ACK] Seq=10928 Ack=80 Win=65536 Len=0 TSval=2335246520...
12	0.001953065	192.168.56.103	192.168.56.102	TCP	66	52400 → 80	[ACK] Seq=80 Ack=10929 Win=84992 Len=0 TSval=270533797 TSecr=...

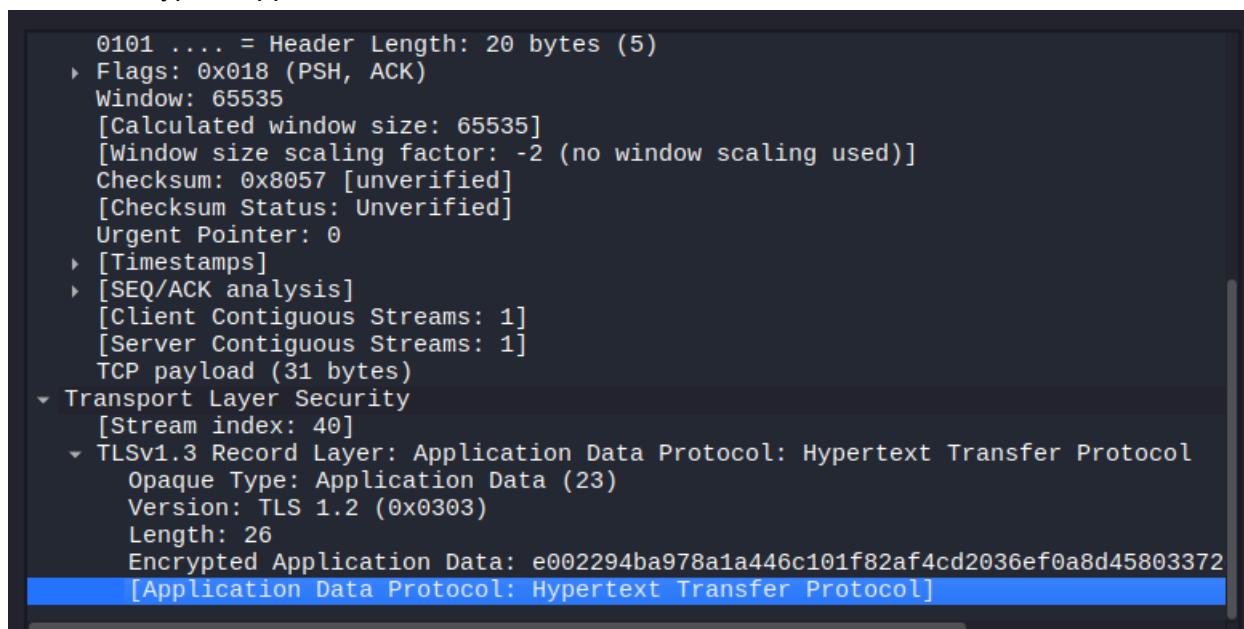
HTTPS

Unlike HTTP, the content of the communication are encrypted. Wireshark can still show information such as source and destination IP address, TLS handshake, certificate, and packet length, but cannot display the actual web content, username, or password, and data being exchanged.

HTTPS using TLS



- TLS client Hello
- TLS server hello
- Certificate exchange
- Encrypted application data



FTP (unsecure protocol)

Run FTP on machine

```
(kali@kali)-[~]
$ sudo ss -tulnp |grep :21
tcp    LISTEN 0      32      0.0.0.0:21      0.0.0.0:21    *:*  users:(("vsftpd",pid=2255
9,fd=3))
```

Generate FTP traffic on KALI

```
Ftp 192.168.56.102
```

Every information inputted appeared on Wireshark no encryption, clear text

Source	Destination	Protocol	Length	Info
192.168.56.102	192.168.56.103	FTP	86	Response: 220 (vsFTPd 3.0.5)
192.168.56.103	192.168.56.102	FTP	86	Request: USER Iamheretotest
192.168.56.102	192.168.56.103	FTP	100	Response: 331 Please specify the password.
192.168.56.103	192.168.56.102	FTP	77	Request: PASS kali
192.168.56.102	192.168.56.103	FTP	88	Response: 530 Login incorrect.

Failed SSH Login Investigation

Generate authentication failure — then investigate

```
(kali@kali)-[~]
$ ssh fakeuser@192.168.56.102
fakeuser@192.168.56.102's password:
Permission denied, please try again.
fakeuser@192.168.56.102's password:
Permission denied, please try again.
fakeuser@192.168.56.102's password:
fakeuser@192.168.56.102: Permission denied (publickey,password).
```

Verify failure on the machine

```
Journalctl -n 50
```

```
connectioniot vsftpd[2155]: pam_unix(vsftpd:auth): check pass; user unknown
connectioniot vsftpd[2155]: pam_unix(vsftpd:auth): authentication failure; logname= uid=0 euid=0 tty=ftp ruser=lamha
connectioniot systemd[1]: Starting sysstat-collect.service - system activity accounting tool...
connectioniot systemd[1]: sysstat-collect.service: Deactivated successfully.
connectioniot systemd[1]: Finished sysstat-collect.service - system activity accounting tool.
connectioniot kernel: workqueue: drm_fb_helper_damage_work hogged CPU for >10000us 4 times, consider switching to WQ
connectioniot sshd-session[2165]: Invalid user fakeuser from 192.168.56.103 port 34814
connectioniot sshd-session[2165]: pam_unix(sshd:auth): check pass; user unknown
connectioniot sshd-session[2165]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser=
connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
connectioniot sshd-session[2165]: pam_unix(sshd:auth): check pass; user unknown
connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
connectioniot sshd-session[2165]: pam_unix(sshd:auth): check pass; user unknown
connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
connectioniot sshd-session[2165]: Connection closed by invalid user fakeuser 192.168.56.103 port 34814 [preauth]
```

Check for failed passwords

```
Journalctl | grep "Failed password"
```

```
connectionuser@connectioniot:~$ journalctl | grep "Failed password"
Jun 01 17:46:21 connectioniot sudo[2228]: connectionuser : TTY=/dev/tty1 ; PWD=/home/connectionuser ; USER=root ; COMMAND=/usr/bin/grep Failed password
Jun 01 17:47:40 connectioniot sudo[2259]: connectionuser : TTY=/dev/tty1 ; PWD=/home/connectionuser ; USER=root ; COMMAND=/usr/bin/grep Failed password
Jun 02 03:04:44 connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
Jun 02 03:04:52 connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
Jun 02 03:04:57 connectioniot sshd-session[2165]: Failed password for invalid user fakeuser from 192.168.56.103 port 34814 ssh2
```

Results

A failed SSH login event was intentionally generated from the Kali Linux VM to the Ubuntu server. Multiple authentication attempts were made using an invalid username and password. The event was investigated using `journalctl` and SSH authentication logs. The source IP, targeted username, and failure status were identified. This demonstrated how security analysts investigate authentication failure alerts and identify potential brute-force activity.

ATTACK— ARP Spoofing (Man in the middle)

Current VMs

-Kali IP: 192.168.56.103

-Ubuntu IP: 192.168.56.102

-windows 11: 192.168.56.101

Install tool on kali

```
Sudo apt update
```

```
Sudo apt install dsniff -y
```

Wireshark filter arp

Generate normal arp ping 192.168.56.102 on kali.

```
valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group def
link/ether 08:00:27:b5:cb:bd brd ff:ff:ff:ff:ff:ff
altname enx080027b5cbbd
inet 192.168.56.102/24 metric 100 brd 192.168.56.255 scope global dynamic enp0s8
valid_lft 375sec preferred_lft 375sec
inet6 fe80::a00:27ff:feb5:cbbd/64 scope link proto kernel_l1
valid_lft forever preferred_lft forever
```

Source	Destination	Protocol	Length	Info
SSystemtec_5c:cf:...	Broadcast	ARP	42	Who has 192.168.56.102? Tell 192.168.56.103
SSystemtec_b5:cb:...	PCSSystemtec_5c:cf:...	ARP	60	192.168.56.102 is at 08:00:27:b5:cb:bd
SSystemtec_b5:cb:...	PCSSystemtec_5c:cf:...	ARP	60	Who has 192.168.56.103? Tell 192.168.56.102
SSystemtec_5c:cf:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.103 is at 08:00:27:5c:cf:6a
SSystemtec_b5:cb:...	PCSSystemtec_5c:cf:...	ARP	60	Who has 192.168.56.103? Tell 192.168.56.102
SSystemtec_5c:cf:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.103 is at 08:00:27:5c:cf:6a

The address resolution protocol was captured in wireshark to observe how devices resolve IP addresses to MAC addresses on the local network.

- ARP Request: “who has 192.168.65.102?”
- ARP Reply: “192.168.56.102 is at 08:00:27:b5:cb:bd”
- Communication is normal now with no interference.

Enable packet forwarding on KALI

```
sudo sysctl -w net.ipv4.ip_forward=1
```

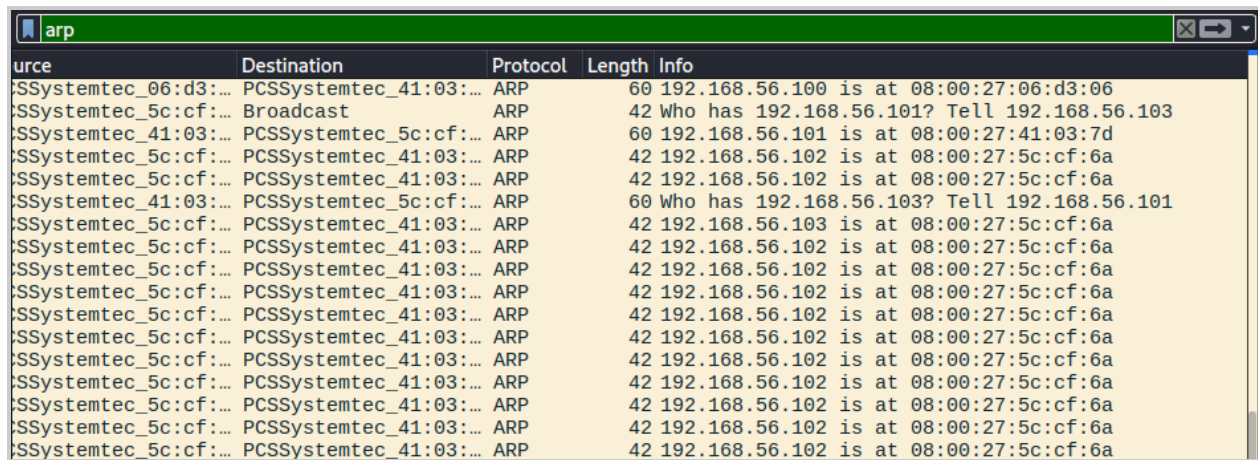
```
(kali㉿kali)-[~]
└─$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
(kali㉿kali)-[~]
└─$ cat /proc/sys/net/ipv4/ip_forward
1
```

Start ARP Spoofing

On Terminal #1

```
sudo arpspoof -i eth1 -t 192.168.56.101 192.168.56.102
```

- Tell windows that Kali is Ubuntu

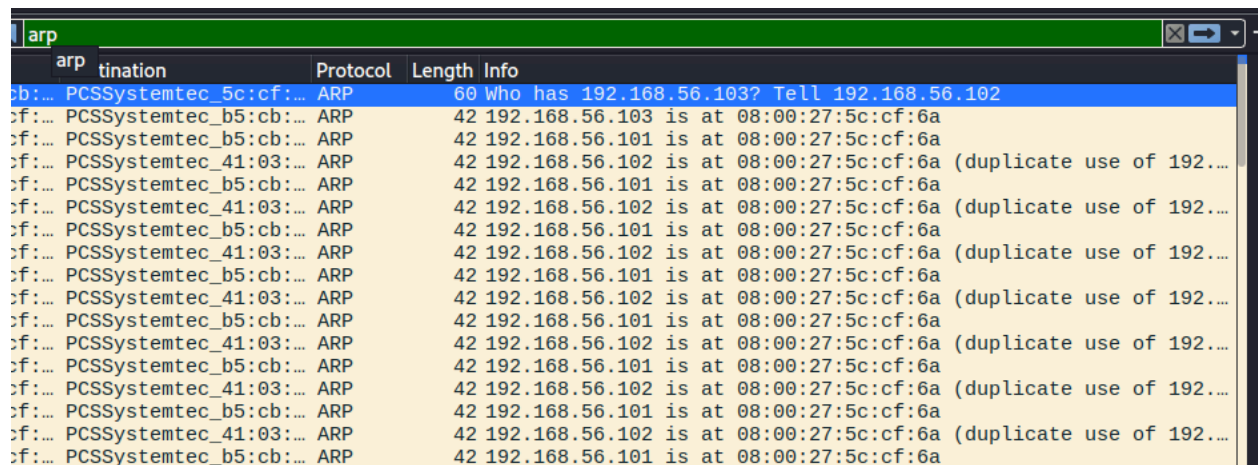


Source	Destination	Protocol	Length	Info
PCSSystemtec_06:d3:...	PCSSystemtec_41:03:...	ARP	60	192.168.56.100 is at 08:00:27:06:d3:06
PCSSystemtec_5c:cf:...	Broadcast	ARP	42	Who has 192.168.56.101? Tell 192.168.56.103
PCSSystemtec_41:03:...	PCSSystemtec_5c:cf:...	ARP	60	192.168.56.101 is at 08:00:27:41:03:7d
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_41:03:...	PCSSystemtec_5c:cf:...	ARP	60	Who has 192.168.56.103? Tell 192.168.56.101
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.103 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a
PCSSystemtec_5c:cf:...	PCSSystemtec_41:03:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a

Terminal #2

```
sudo arpspoof -i eth1 -t 192.168.56.102 192.168.56.101
```

- Tell Ubuntu that Kali is Windows



Source	Destination	Protocol	Length	Info
PCSSystemtec_b5:cb:...	PCSSystemtec_41:03:...	ARP	60	who has 192.168.56.103? Tell 192.168.56.102
PCSSystemtec_b5:cb:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.103 is at 08:00:27:5c:cf:6a
PCSSystemtec_b5:cb:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
PCSSystemtec_41:03:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192...)
PCSSystemtec_b5:cb:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
PCSSystemtec_41:03:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192...)
PCSSystemtec_b5:cb:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
PCSSystemtec_41:03:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192...)
PCSSystemtec_b5:cb:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
PCSSystemtec_41:03:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192...)
PCSSystemtec_b5:cb:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
PCSSystemtec_41:03:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192...)
PCSSystemtec_b5:cb:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a
PCSSystemtec_41:03:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.102 is at 08:00:27:5c:cf:6a (duplicate use of 192...)
PCSSystemtec_b5:cb:...	PCSSystemtec_b5:cb:...	ARP	42	192.168.56.101 is at 08:00:27:5c:cf:6a

Observation

- Multiple replies claiming ownership indicated ARP poisoning, for both Terminals.
- Duplicate IP warning (wireshark detects conflicting ARP information and flags it as duplicate).

```

Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\zaina>arp -a

Interface: 10.0.2.15 --- 0x2
Internet Address      Physical Address      Type
10.0.2.2              52-54-00-12-35-00    dynamic
10.0.2.255           ff-ff-ff-ff-ff-ff    static
224.0.0.22           01-00-5e-00-00-16    static
224.0.0.252          01-00-5e-00-00-fc    static
239.255.255.250      01-00-5e-7f-ff-fa    static
255.255.255.255      ff-ff-ff-ff-ff-ff    static

Interface: 192.168.56.101 --- 0x7
Internet Address      Physical Address      Type
192.168.56.100       08-00-27-06-d3-06    dynamic
192.168.56.103       08-00-27-5c-cf-6a    dynamic
192.168.56.255       ff-ff-ff-ff-ff-ff    static
224.0.0.22           01-00-5e-00-00-16    static
224.0.0.252          01-00-5e-00-00-fc    static
239.255.255.250      01-00-5e-7f-ff-fa    static
255.255.255.255      ff-ff-ff-ff-ff-ff    static

C:\Users\zaina>
C:\Users\zaina>

```

Checking ARP table on windows 11 shows that the it was changed to Kali's MAC address

```

3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5c:cf:6a brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
        valid_lft 402sec preferred_lft 402sec
    inet6 fe80::e178:c753:f158:ce5b/64 scope link noprefixroute
    valid_lft forever preferred_lft forever

```

Detection summary:

An ARP spoofing attack was simulated in this slab using Kali Linux. The attacker system sent forged ARP replies to both the Windows and Ubuntu Hosts, attempting to associate the attacker's MAC address with legitimate addresses.

Findings

- Repeated ARP replies were observed in Wireshark.
- ARP cache entries on the victim system (windows 11) changed during the attack)
- The attacker attempted to position itself between the two communicating hosts (Ubuntu and windows 11).

Indicators of compromise:

- Multiple ARP replies without corresponding requests.
- IP addresses resolving to unexpected MAC addresses.
- Frequent ARP updates in a short time period.

Last DEMO showing ARP table before and right after spoofing attack
WINDOWS 11
BEFORE

```
C:\Users\zaina>arp -a

Interface: 10.0.2.15 --- 0x2
    Internet Address      Physical Address      Type
    10.0.2.2              52-54-00-12-35-00    dynamic
    10.0.2.255            ff-ff-ff-ff-ff-ff    static
    224.0.0.22            01-00-5e-00-00-16    static
    224.0.0.252          01-00-5e-00-00-fc    static
    239.255.255.250       01-00-5e-7f-ff-fa    static
    255.255.255.255       ff-ff-ff-ff-ff-ff    static

Interface: 192.168.56.101 --- 0x7
    Internet Address      Physical Address      Type
    192.168.56.100        08-00-27-06-d3-06    dynamic
    192.168.56.102        08-00-27-b5-cb-bd    dynamic
    192.168.56.103        08-00-27-5c-cf-6a    dynamic
    192.168.56.255        ff-ff-ff-ff-ff-ff    static
    224.0.0.22            01-00-5e-00-00-16    static
    224.0.0.252          01-00-5e-00-00-fc    static
    239.255.255.250       01-00-5e-7f-ff-fa    static
    255.255.255.255       ff-ff-ff-ff-ff-ff    static
```

AFTER

```
C:\Users\zaina>arp -a

Interface: 10.0.2.15 --- 0x2
    Internet Address      Physical Address      Type
    10.0.2.2              52-54-00-12-35-00    dynamic
    10.0.2.255            ff-ff-ff-ff-ff-ff    static
    224.0.0.22            01-00-5e-00-00-16    static
    224.0.0.252          01-00-5e-00-00-fc    static
    239.255.255.250       01-00-5e-7f-ff-fa    static
    255.255.255.255       ff-ff-ff-ff-ff-ff    static

Interface: 192.168.56.101 --- 0x7
    Internet Address      Physical Address      Type
    192.168.56.100        08-00-27-06-d3-06    dynamic
    192.168.56.102        08-00-27-5c-cf-6a    dynamic
    192.168.56.103        08-00-27-5c-cf-6a    dynamic
    192.168.56.255        ff-ff-ff-ff-ff-ff    static
    224.0.0.22            01-00-5e-00-00-16    static
    224.0.0.252          01-00-5e-00-00-fc    static
    239.255.255.250       01-00-5e-7f-ff-fa    static
    255.255.255.255       ff-ff-ff-ff-ff-ff    static
```


ATTACK — SYNC flood detection

TCP normal connection looks like

SYN

SYN-ACK

ACK

Wireshark filter `tcp.flags.syn == 1`

Kali Linux install

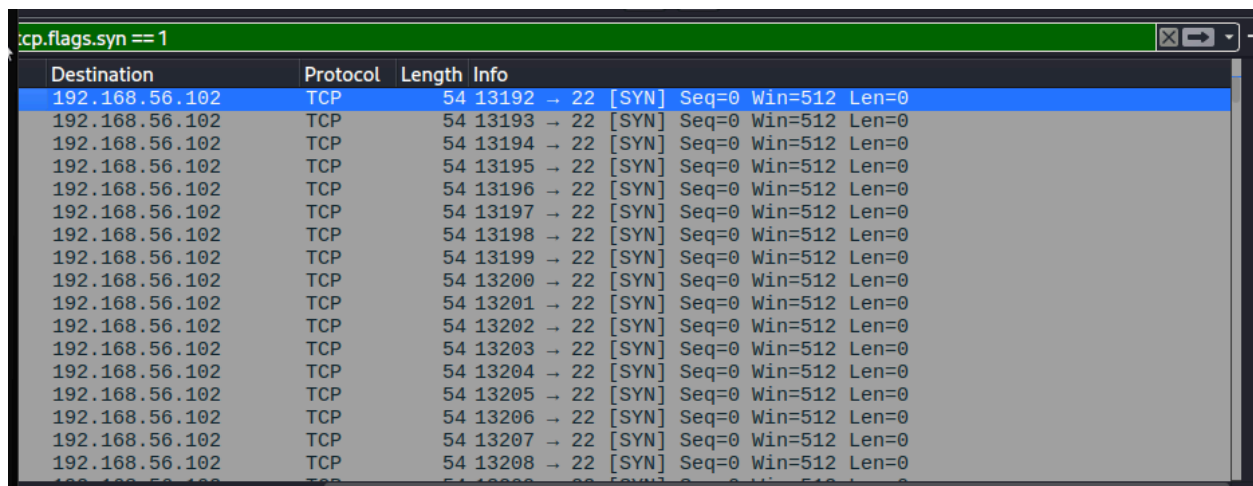
```
Sudo hping3 -S -p 22 -flood 192.168.56.102
```

(to generate flood)

-S = SYN flag

-p 22 = target SSH

-flood = send rapidly



Destination	Protocol	Length	Info
192.168.56.102	TCP	54	13192 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13193 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13194 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13195 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13196 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13197 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13198 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13199 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13200 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13201 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13202 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13203 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13204 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13205 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13206 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13207 → 22 [SYN] Seq=0 Win=512 Len=0
192.168.56.102	TCP	54	13208 → 22 [SYN] Seq=0 Win=512 Len=0

Documentation:

A SYN flood attack was simulated in the lab environment using hping3. The attack generated a large number of TCP SYN packets toward the SSH service running on the Ubuntu server. Wireshark was used to monitor the traffic and identify abnormal connection behavior.

Observations:

- Thousands of TCP SYN packets were observed causing my whole windows 11 to become slow. It is also a compromise of a DDOS attack.

- Connections were initiated but not completed (SYN, SYN-ACK, ACK)
- Excessive connection attempts occurred within a short time period.

Indicators of compromise

- Large volume of SYN packets from a single source
- Repeated connection attempts without successful session establishment
- Resource exhaustion on the target system.

ATTACK — DNS Spoofing/ DNS Poisoning detection

This will explain

- Fake websites
- Phishing
- Malware redirection
- DNS hijacking

SAFE DNS poisoning Simulation

Verifying that Ubuntu server is running a DNS service

```
Sudo ss -tulnp | grep :53
```

1. Create my own fake domain

Create a zone file

Called: **lab.local**

```
Sudo nano /etc/bind/db.lab.local
```

```
connectionuser@connectioniot:~$ cat /etc/bind/db.lab.local
$TTL 604800
@ IN SOA nsl.lab.local. root.lab.local. (
1
604800
86400
2419200
604800
)

@ IN NS nsl.lab.local.
nsl IN A 192.168.56.102
bank IN A 192.168.56.103
server IN A 192.168.56.102
```

Create Configuration File

```
sudo nano /etc/bind/named.conf.local
```

```
connectionuser@connectioniot:~$ cat /etc/bind/named.conf.local
zone "lab.local" {
type master;
file "/etc/bind/db.lab.local";
};
```

Verify files

```
connectionuser@connectioniot:~$ sudo named-checkzone lab.local /etc/bind/db.lab.local
zone lab.local/IN: loaded serial 1
OK
```

Restart and check

```
connectionuser@connectioniot:~$ sudo systemctl restart bind9
connectionuser@connectioniot:~$ nslookup bank.lab.local 192.168.56.102
Server:          192.168.56.102
Address:         192.168.56.102#53

Name:   bank.lab.local
Address: 192.168.56.103
```

The goal of this lab was to understand how DNS records can influence where users are directed when they visit a website. This demonstrates the core concept behind DNS poisoning attacks, where an attacker manipulates DNS responses to redirect users to unintended systems.

Instead of targeting real websites, a controlled lab environment was used to safely simulate the behavior.

What Normal looks like

User



DNS Query



Legitimate Website

Poisoned response looks like

User



DNS Query



Attacker-Controlled System

NOW THAT WE created a domain lets query from Kali the process is the victims would think that they are accessing a bank server but really attackers tricks DNS and instead the attacker server not bank server.

Query on Kali

```
(kali㉿kali)-[~]  
$ nslookup bank.lab.local 192.168.56.102  
Server:      192.168.56.102  
Address:     192.168.56.102#53  
  
Name:   bank.lab.local  
Address: 192.168.56.103
```

Wireshark captures

	Source	Destination	Protocol	Length	Info
7	192.168.56.103	192.168.56.102	DNS	74	Standard query 0x6981 A bank.lab.local
7	192.168.56.102	192.168.56.103	DNS	90	Standard query response 0x6981 A bank.lab.local A 192.168.56.103
5	192.168.56.103	192.168.56.102	DNS	74	Standard query 0x6beb AAAA bank.lab.local
2	192.168.56.102	192.168.56.103	DNS	119	Standard query response 0x6beb AAAA bank.lab.local SOA ns1.lab.local

Basic summary

Whenever someone asks for bank.lab.local, send them to Kali.

if Kali was running a fake banking website, the victim would end up on Kali instead of the real server.

Note it would look like a legit website just like a banking website to trick the victim.

Indicators of compromise:

- DNS response contains unexpected IP addresses. (not matching theirs)
- internal/private IPs returned for public websites.
- Multiple conflicting DNS responses for the same domain
- Immediate follow of suspicious DNS responses.